Top 30 LeetCode SQL Interview Questions – Part 2

MySQL flavor — Curated for interview practice

Curated by Tarun Reddy

Data Manipulation & Exchange

1. Exchange Seats

Explanation: Swap student seats where odd/even seats are exchanged.

```
-- Exchange Seats (MySQL)
SELECT s.StudentId, CASE WHEN s.Seat%2=1 THEN s.Seat+1 ELSE s.Seat-1 END AS ReassignedSeat
FROM Seats s
ORDER BY s.StudentId;
```

2. Fix Names in Table

Explanation: Standardize names: trim spaces and title-case basic capitalization.

```
-- Fix names (basic normalization)
UPDATE Persons
SET Name = CONCAT(UPPER(SUBSTRING(TRIM(Name),1,1)), LOWER(SUBSTRING(TRIM(Name),2)))
WHERE Name IS NOT NULL;
```

3. Replace Employee ID With Unique Identifier

Explanation: Generate deterministic unique identifier using EmployeeId.

```
-- Create Unique Identifier (display)
SELECT EmployeeId, CONCAT('EMP-', LPAD(EmployeeId,6,'0')) AS EmpUID, Name
FROM Employee;
```

Projects & Participation

4. Project Employees I

Explanation: Return employees who never participated in any project.

```
-- Project Employees I
SELECT e.EmployeeId, e.Name
FROM Employees e
LEFT JOIN ProjectParticipation p ON e.EmployeeId = p.EmployeeId
WHERE p.EmployeeId IS NULL;
```

5. Project Employees II

Explanation: List employees who participated in every project.

```
-- Project Employees II
SELECT p.EmployeeId
FROM ProjectParticipation p
GROUP BY p.EmployeeId
HAVING COUNT(DISTINCT p.ProjectId) = (SELECT COUNT(*) FROM Projects);
```

6. Project Employees III

Explanation: For each project, list employees assigned but who didn't submit reports.

```
-- Project Employees III
SELECT a.ProjectId, a.EmployeeId
FROM Assignments a
LEFT JOIN Reports r ON a.ProjectId = r.ProjectId AND a.EmployeeId = r.EmployeeId
WHERE r.ReportId IS NULL;
```

Sales & Products

7. Product Sales Analysis I

Explanation: Find total sales per product (quantity * price) and rank.

```
-- Product Sales Analysis I
SELECT p.ProductId, p.ProductName, SUM(o.Quantity * p.Price) AS TotalSales
FROM Products p
JOIN Orders o ON p.ProductId = o.ProductId
GROUP BY p.ProductId, p.ProductName
ORDER BY TotalSales DESC;
```

8. Product Sales Analysis II

Explanation: Top product each month by sales amount.

```
-- Product Sales Analysis II
SELECT t.Year, t.Month, t.ProductId, t.TotalSales
FROM (
  SELECT YEAR(o.OrderDate) AS Year, MONTH(o.OrderDate) AS Month, o.ProductId,
         SUM(o.Quantity * p.Price) AS TotalSales,
         DENSE_RANK() OVER (PARTITION BY YEAR(o.OrderDate), MONTH(o.OrderDate) ORDER BY SUM(o.Quant
AS rk
  FROM Orders o JOIN Products p ON o.ProductId = p.ProductId
  GROUP BY YEAR(o.OrderDate), MONTH(o.OrderDate), o.ProductId
) t
WHERE t.rk = 1;
```

9. Product Sales Analysis III

Explanation: Products with increasing sales for three consecutive months.

```
-- Product Sales Analysis III
WITH monthly AS (
  SELECT p.ProductId, DATE_FORMAT(o.OrderDate,'%Y-%m') AS ym,
         SUM(o.Quantity*p.Price) AS sales
  FROM Orders o JOIN Products p ON o.ProductId=p.ProductId
  GROUP BY p.ProductId, ym
)
SELECT DISTINCT m1.ProductId
FROM monthly m1
JOIN monthly m2 ON m1.ProductId=m2.ProductId AND DATE_ADD(STR_TO_DATE(m1.ym,'%Y-%m'), INTERVAL 1 MO
STR_TO_DATE(m2.ym,'%Y-%m')
JOIN monthly m3 ON m2.ProductId=m3.ProductId AND DATE_ADD(STR_TO_DATE(m2.ym,'%Y-%m'), INTERVAL 1 MO
STR_TO_DATE(m3.ym,'%Y-%m')
WHERE m1.sales < m2.sales AND m2.sales < m3.sales;
```

Customers & Orders

10. Customers Bought Same Day Twice

Explanation: Find customers who placed more than one order on the same date.

```
-- Customers with multiple orders on same day
SELECT CustomerId, OrderDate
FROM (
  SELECT CustomerId, DATE(OrderDate) AS OrderDate, COUNT(*) AS cnt
  FROM Orders
  GROUP BY CustomerId, DATE(OrderDate)
) t
WHERE t.cnt > 1;
```

11. Customers Who Bought All Products

Explanation: Customers who purchased every product at least once.

Customers & Orders (contd.)

```
-- Customers who bought all products
SELECT o.CustomerId
FROM Orders o
GROUP BY o.CustomerId
HAVING COUNT(DISTINCT o.ProductId) = (SELECT COUNT(*) FROM Products);
```

12. Percentage of Returning Customers

Explanation: Percentage of customers with >1 purchase in a year.

```
-- Percentage of returning customers in a year
WITH yearly AS (
  SELECT CustomerId, COUNT(DISTINCT OrderId) AS orders
  FROM Orders
  WHERE YEAR(OrderDate)=2023
  GROUP BY CustomerId
)
SELECT ROUND(100 * SUM(CASE WHEN orders>1 THEN 1 ELSE 0 END) / COUNT(*),2) AS pct_returning
FROM yearly;
```

Time Series & Growth

13. Restaurant Growth

Explanation: Restaurants with higher revenue this year vs last year.

```
-- Restaurant growth year-over-year
WITH rev AS (
  SELECT RestaurantId, YEAR(OrderDate) AS yr, SUM(Amount) AS revenue
  FROM Orders
  GROUP BY RestaurantId, YEAR(OrderDate)
)
SELECT curr.RestaurantId
FROM rev curr
JOIN rev prev ON curr.RestaurantId = prev.RestaurantId AND curr.yr = prev.yr + 1
WHERE curr.revenue > prev.revenue;
```

14. Customers With Increasing Spend (3 months)

Explanation: Customers whose spend increased every month for 3 consecutive months.

```
-- Customers with increasing spend over 3 consecutive months
WITH monthly AS (
  SELECT CustomerId, DATE_FORMAT(OrderDate,'%Y-%m') AS ym, SUM(Amount) AS spend
  FROM Orders
  GROUP BY CustomerId, ym
)
SELECT DISTINCT m1.CustomerId
FROM monthly m1
JOIN monthly m2 ON m1.CustomerId=m2.CustomerId AND DATE_ADD(STR_TO_DATE(m1.ym,'%Y-%m'), INTERVAL 1
MONTH)=STR_TO_DATE(m2.ym,'%Y-%m')
JOIN monthly m3 ON m2.CustomerId=m3.CustomerId AND DATE_ADD(STR_TO_DATE(m2.ym,'%Y-%m'), INTERVAL 1
MONTH)=STR_TO_DATE(m3.ym,'%Y-%m')
WHERE m1.spend < m2.spend AND m2.spend < m3.spend;
```

Analytics & Window Functions

15. Orders With Top 3 Quantity Per Product

Explanation: For each product find orders that are in the top 3 quantities.

Analytics & Window Functions (contd.)

```
-- Orders in top 3 quantities per product
SELECT ProductId, OrderId, Quantity
FROM (
  SELECT o.ProductId, o.OrderId, o.Quantity,
         DENSE_RANK() OVER (PARTITION BY o.ProductId ORDER BY o.Quantity DESC) AS rk
  FROM Orders o
) t
WHERE rk <= 3;
```

16. Peak Hour Orders

Explanation: Find hour with max orders for each day.

```
-- Peak hour per day
SELECT OrderDate, HOUR(OrderTime) AS hr, cnt FROM (
  SELECT DATE(OrderDate) AS OrderDate, HOUR(OrderTime) AS hour, COUNT(*) AS cnt,
         ROW_NUMBER() OVER (PARTITION BY DATE(OrderDate) ORDER BY COUNT(*) DESC) AS rn
  FROM Orders
  GROUP BY DATE(OrderDate), HOUR(OrderTime)
) t WHERE rn = 1;
```

17. Find Cumulative Salary of an Employee

Explanation: Calculate running salary totals using window functions.

```
-- Cumulative salary
SELECT Id, Month, Salary, SUM(Salary) OVER (PARTITION BY Id ORDER BY Month) AS CumulativeSalary
FROM Employee;
```

User Activity & Logs

18. Find Users with Consecutive Logins (k days)

Explanation: Users who logged in for at least k consecutive days (example k=3).

```
-- Users with at least 3 consecutive login days
WITH d AS (
  SELECT user_id, login_date,
         DATE_SUB(login_date, INTERVAL ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date)
  FROM Logins
  GROUP BY user_id, login_date
)
SELECT user_id
FROM d
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

19. Game Play Analysis I - First Login

Explanation: Find the first login date for each player.

```
-- First login per player
SELECT player_id, MIN(event_date) AS first_login
FROM Activity
GROUP BY player_id;
```

20. Game Play Analysis II - First Login Device

Explanation: Get player ID and device ID of first login for each player.

User Activity & Logs (contd.)

```
-- First login device per player
SELECT a.player_id, a.device_id
FROM Activity a
WHERE (a.player_id, a.event_date) IN (
  SELECT player_id, MIN(event_date)
  FROM Activity
  GROUP BY player_id
);
```

Anomaly Detection & Sequences

21. Consecutive Days with High Traffic

Explanation: Sites with at least 7 consecutive days where visits exceed threshold.

```
-- Sites with 7 consecutive days above threshold
WITH flagged AS (
  SELECT SiteId, VisitDate,
         CASE WHEN Visits > 10000 THEN 1 ELSE 0 END AS hot
  FROM SiteVisits
),
grp AS (
  SELECT SiteId, VisitDate,
         DATE_SUB(VisitDate, INTERVAL ROW_NUMBER() OVER (PARTITION BY SiteId ORDER BY VisitDate) DA
  FROM flagged WHERE hot=1
)
SELECT SiteId
FROM grp
GROUP BY SiteId, grp
HAVING COUNT(*) >= 7;
```

22. Human Traffic of Stadium

Explanation: Return records where people count was >=100 for 3+ consecutive days.

```
-- Stadium consecutive high people
SELECT Id FROM (
  SELECT Id, Lead(People,1) OVER (ORDER BY Id) AS next1, Lead(People,2) OVER (ORDER BY Id) AS next2
  FROM Stadium
  WHERE People >= 100
) t
WHERE next1 >= 100 AND next2 >= 100;
```

Text & Reviews

23. Top K Frequent Words in Reviews

Explanation: Find most frequent words in review text (approximation using split).

```
-- Top K words (approximation)
SELECT word, COUNT(*) AS freq FROM (
  SELECT LOWER(SUBSTRING_INDEX(SUBSTRING_INDEX(Content, ' ', n.n), ' ', -1)) AS word
  FROM Reviews
  JOIN (SELECT 1 as n UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 U
UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9 UNION ALL SELECT 10) n
  WHERE CHAR_LENGTH(Content) - CHAR_LENGTH(REPLACE(Content,' ','') ) >= n.n-1
) t
WHERE word <> ''
GROUP BY word
ORDER BY freq DESC
LIMIT 20;
```

24. Find Cumulative Average Sessions Per User

Explanation: Compute average sessions per user by counting distinct event dates.

```
-- Avg sessions per user (example)
SELECT player_id, ROUND(COUNT(DISTINCT event_date) / COUNT(DISTINCT login_date),2) AS avg_sessions_
FROM Activity
GROUP BY player_id;
```

Inventory & Operations

25. Inventory Reorder List

Explanation: Products with stock below threshold and average daily sales to compute reorder qty.

```
-- Inventory reorder suggestion
SELECT p.ProductId, p.Stock, ROUND(AVG(daily.sales),2) AS avg_daily_sales,
       GREATEST(0, CEIL((30*ROUND(AVG(daily.sales),2) - p.Stock))) AS reorder_qty
FROM Products p
LEFT JOIN (
  SELECT ProductId, DATE(OrderDate) AS d, SUM(Quantity) AS sales
  FROM Orders
  GROUP BY ProductId, DATE(OrderDate)
) daily ON p.ProductId = daily.ProductId
GROUP BY p.ProductId, p.Stock
HAVING p.Stock < 30*ROUND(AVG(daily.sales),2);
```

26. Inventory - Products Never Sold

Explanation: List products that have zero orders.

```
-- Products never sold
SELECT p.ProductId, p.ProductName
FROM Products p
LEFT JOIN Orders o ON p.ProductId = o.ProductId
WHERE o.OrderId IS NULL;
```

Maintenance & Management

27. Remove Inactive Users Older Than N Days

Explanation: Delete users who haven't logged in for N days (example N=365).

```
-- Delete inactive users
DELETE FROM Users
WHERE LastLogin < DATE_SUB(CURDATE(), INTERVAL 365 DAY);
```

28. Update Prices by Percentage

Explanation: Increase prices of a category by X percent in a single statement.

```
-- Bulk price update by percent
UPDATE Products
SET Price = ROUND(Price * (1 + 0.12), 2)
WHERE Category = 'Electronics';
```

29. Orders With Top 3 Quantity Per Product (Alternate)

Explanation: Another form to get top orders per product using ROW_NUMBER window.

Maintenance & Management (contd.)

```
-- Top orders per product using ROW_NUMBER
SELECT ProductId, OrderId, Quantity FROM (
  SELECT o.ProductId, o.OrderId, o.Quantity,
         ROW_NUMBER() OVER (PARTITION BY o.ProductId ORDER BY o.Quantity DESC) AS rn
  FROM Orders o
) t WHERE rn <= 3;
```

Misc & Utilities

30. Products Bundled Together

Explanation: Pairs of products frequently bought together (support > threshold).

```
-- Product pairs bought together (co-occurrence)
SELECT a.ProductId AS p1, b.ProductId AS p2, COUNT(*) AS cnt
FROM Orders a
JOIN Orders b ON a.OrderGroupId = b.OrderGroupId AND a.ProductId < b.ProductId
GROUP BY a.ProductId, b.ProductId
HAVING cnt >= 10
ORDER BY cnt DESC;
```